

Module 1: Introduction to Programming & Problem Solving

Case Studies & Learner Takeaways

Duration: 4 Hours

Level: Beginner

Programming Language: C

Module Overview

This module introduces learners to the fundamentals of computational thinking and programming. Students will learn how to analyze real-world problems, design algorithms, create flowcharts and pseudocode, understand basic time and space complexity, and implement simple C programs using input and output operations.

Module Topics

- Introduction to Algorithms and Data Structures
 - Problem-Solving Methodology
 - Flowcharts and Pseudocode
 - Time and Space Complexity Basics
 - Introduction to C Programming Environment
 - Input/Output Operations
-

Case Study 1: Smart ATM Cash Withdrawal System

Business Scenario

ABC Bank plans to automate its ATM operations. Before implementation, the software development team must design a solution using computational thinking techniques.

The ATM should:

- Accept the customer's PIN.
- Verify whether the PIN is correct.
- Ask for the withdrawal amount.
- Check whether sufficient balance is available.
- Dispense cash if all conditions are satisfied.
- Display appropriate error messages for invalid PIN or insufficient balance.

Learning Objectives

- Understand problem decomposition.
- Design step-by-step algorithms.
- Create flowcharts and pseudocode.
- Identify inputs, outputs, and processing steps.
- Discuss algorithm efficiency.

Student Activities

1. Identify the problem statement.
2. List inputs, outputs, and processing logic.
3. Write an algorithm.
4. Draw a flowchart.
5. Write pseudocode.
6. Implement a basic C program using `printf()` and `scanf()` (decision logic can be demonstrated if covered).

Expected Deliverables

- Algorithm
- Flowchart
- Pseudocode
- Basic C Program
- Complexity discussion

Discussion Questions

- What happens if the customer enters an incorrect PIN?
- Can there be more than one correct algorithm for the same problem?

- What is the time complexity of PIN verification?
-

Case Study 2: College Student Result Processing System

Business Scenario

XYZ University wants to automate student result processing by calculating the total marks, average, and percentage for each student.

The application should:

- Accept marks for five subjects.
- Calculate the total marks.
- Calculate the average.
- Calculate the percentage.
- Display the result summary.

Learning Objectives

- Use variables and arithmetic operations.
- Understand structured problem solving.
- Apply algorithms and pseudocode.
- Practice user input and formatted output.

Sample Input

Math = 80

Science = 75

English = 90

Computer = 95

Physics = 85

Expected Output

Total = 425

Average = 85

Percentage = 85%

Student Activities

- Write the algorithm.
- Draw the flowchart.
- Prepare pseudocode.
- Develop the C program.
- Analyze the time complexity.

Complexity Analysis

- Time Complexity: **$O(1)$**
 - Space Complexity: **$O(1)$**
-

Case Study 3: Smart Grocery Billing System

Business Scenario

A supermarket wants to automate its billing process. The cashier enters the product price and quantity, and the system calculates the final amount.

Functional Requirements

- Accept product price.
- Accept quantity.
- Calculate item total.
- Display the final bill.

Future enhancements may include:

- GST calculation
- Discount calculation
- Multiple products

Learning Objectives

- Formula implementation
- Arithmetic operations
- Input and output handling
- Algorithm design

Sample Input

Price = 60

Quantity = 5

Expected Output

Total Bill = 300

Student Activities

- Identify problem components.
- Develop an algorithm.
- Draw a flowchart.
- Write pseudocode.
- Implement the solution in C.

Classroom Discussion

How would the algorithm change if multiple products were added to the bill?

Case Study 4: Smart Weather Temperature Converter

Business Scenario

A weather application stores temperatures in Celsius, while international users require values in Fahrenheit.

The application converts temperatures using the standard mathematical formula.

Formula

$$\text{Fahrenheit} = (\text{Celsius} \times 9 / 5) + 32$$

Learning Objectives

- Arithmetic expressions
- Variables and data types
- Formula implementation

- Input and output operations

Sample Input

Celsius = 30

Expected Output

Fahrenheit = 86

Student Activities

- Develop an algorithm.
- Draw a flowchart.
- Write pseudocode.
- Implement the solution in C.

Complexity Analysis

- Time Complexity: **$O(1)$**
- Space Complexity: **$O(1)$**

Discussion

Why is this considered a constant-time operation?

Case Study 5: Library Book Search Optimization

Business Scenario

A college library maintains over 10,000 books. The librarian wants to search books efficiently.

Two approaches are proposed:

Approach 1

Search each book one by one (Linear Search).

Approach 2

Arrange books alphabetically and repeatedly divide the search space (Binary Search).

Students analyze both approaches and recommend the better solution.

Learning Objectives

- Algorithm comparison
- Logical thinking
- Time complexity analysis
- Introduction to search algorithms

Classroom Activity

Arrange students according to roll numbers.

One student performs:

- Linear Search

Another student performs:

- Binary Search

Compare:

- Number of comparisons
- Time taken
- Efficiency

Complexity Comparison

Algorithm	Best Case	Worst Case	Space Complexity
Linear Search	$O(1)$	$O(n)$	$O(1)$
Binary Search	$O(1)$	$O(\log n)$	$O(1)$

Discussion Questions

- Why is Binary Search faster?
 - Can Binary Search work on unsorted data?
 - Which search approach is suitable for large-scale applications?
-

Module-End Mini Project

Smart Student Utility Application

Learners will integrate multiple concepts from this module by designing a simple console-based application.

Functional Requirements

- Accept student details.
 - Calculate total, average, and percentage.
 - Calculate the area of a classroom.
 - Convert classroom temperature from Celsius to Fahrenheit.
 - Display formatted output.
 - Document the solution using an algorithm, flowchart, and pseudocode.
-

Learner Takeaways

By the end of this module, learners will be able to:

Programming Fundamentals

- Explain the purpose and importance of programming.
- Understand the role of algorithms in software development.
- Differentiate between algorithms, programs, compilers, and programming languages.

Problem Solving

- Analyze real-world problems systematically.
- Identify inputs, outputs, and processing steps.
- Break large problems into smaller, manageable tasks.

Algorithm Design

- Write clear and structured algorithms.
- Compare alternative solutions for the same problem.
- Evaluate the correctness and efficiency of algorithms.

Flowcharts and Pseudocode

- Use standard flowchart symbols.
- Create flowcharts for simple applications.
- Write pseudocode before coding.
- Translate algorithms into program logic.

Time and Space Complexity

- Explain why algorithm efficiency matters.
- Understand the basics of Big-O notation.
- Compare constant, linear, and logarithmic time complexity.
- Recognize situations where efficient algorithms improve performance.

C Programming Environment

- Install and configure Visual Studio Code.
- Configure the GCC compiler.
- Compile and execute C programs successfully.
- Understand the compilation process.

Input and Output Operations

- Use `printf()` to display output.
- Use `scanf()` to accept user input.
- Apply appropriate format specifiers.
- Understand the purpose of the address (&) operator in `scanf()`.

Variables and Data Types

- Declare and initialize variables.
- Work with integer, floating-point, character, and string data types.
- Perform arithmetic operations using variables.

Basic Program Development

Develop simple applications such as:

- Hello World
- Student Information System
- Calculator

- Area Calculator
- Temperature Converter
- Grocery Billing System
- Student Marks Calculator
- ATM Simulation

Debugging Skills

- Identify syntax errors.
- Interpret compiler error messages.
- Correct common programming mistakes.
- Develop confidence in troubleshooting code.

Computational Thinking

Learners will strengthen their ability to:

- Think logically.
 - Analyze problems critically.
 - Design structured solutions.
 - Improve algorithm efficiency.
 - Write clean and maintainable code.
-

Practical Skills Gained

After completing this module, learners will be able to:

- Set up a C programming environment.
 - Write, compile, execute, and debug C programs.
 - Design algorithms and flowcharts.
 - Create pseudocode for programming problems.
 - Implement arithmetic-based applications.
 - Analyze the efficiency of basic algorithms.
 - Apply structured problem-solving techniques.
-

Professional Skills Developed

This module also helps learners build essential workplace skills, including:

- Analytical Thinking
 - Logical Reasoning
 - Critical Thinking
 - Computational Thinking
 - Attention to Detail
 - Debugging Mindset
 - Communication through Structured Documentation
 - Team-based Problem Solving
-

Assessment Rubric

Evaluation Criteria	Marks
Problem Understanding	10
Algorithm Design	20
Flowchart Accuracy	15
Pseudocode Quality	15
C Program Implementation	25
Code Readability & Documentation	5
Time & Space Complexity Analysis	10
Total	100

Module Completion Checklist

At the end of this module, learners should be able to:

- ☒ Explain the importance of programming and algorithms.
- ☒ Apply structured problem-solving techniques.
- ☒ Design algorithms for simple computational problems.
- ☒ Create flowcharts using standard symbols.
- ☒ Develop pseudocode before coding.

-  Explain basic time and space complexity concepts.
 -  Configure and use the C programming environment.
 -  Write, compile, execute, and debug basic C programs.
 -  Use `printf()` and `scanf()` effectively.
 -  Perform arithmetic calculations using variables.
 -  Analyze simple algorithms using Big-O notation.
 -  Solve beginner-level programming problems independently.
-

Conclusion

Module 1 establishes the foundation for every programming concept that follows. Through real-world case studies, hands-on coding exercises, algorithms, flowcharts, pseudocode, and complexity analysis, learners develop the computational thinking and practical programming skills required for software development. These competencies prepare them to confidently progress to advanced topics such as decision-making, loops, functions, arrays, pointers, and data structures in subsequent modules.